

## **Manual Técnico:**

## **Introducción:**

El siguiente manual tiene como fin dar una explicación sobre el código que se realizó en el lenguaje Java, se podrá ver una estructura del orden que se usó, así como la forma en la que se programó el analizador léxico.

```

Lexer.flex x BattleEngine.java [-/M] x MainFrame.java x PropertyEntry.java [-/M] x Parser.cup [-/M] x
Source #flex History
1 package analyzer;
2
3 import java_cup.runtime.Symbol;
4 import java.util.ArrayList;
5 import java.util.List;
6 import model.Action;
7
8
9
10 %public
11 %class Lexer
12 %unicode
13 %cup
14 %line
15 %column
16 %state COMMENT
17
18 %{
19
20 private final List<TokenInfo> tokens = new ArrayList<>();
21 private final List<CompileError> errors = new ArrayList<>();
22
23 private int commentLine;
24 private int commentColumn;
25
26 public List<TokenInfo> getTokens() {
27     return tokens;
28 }
29
30 public List<CompileError> getErrors() {
31     return errors;
32 }
33
34 private Symbol token(int id, String type) {
35     String lexeme = yytext();
36
37     tokens.add(new TokenInfo(
38         tokens.size() + 1,
39         lexeme,
40         type,
41         yyline + 1,
42         yycolumn + 1
43     ));
44
45     return new Symbol(
46         id,

```

```

Lexer.flex x BattleEngine.java [-/M] x MainFrame.java x PropertyEntry.java [-/M] x Parser.cup [-/M] x
Source #flex History
52
53 private Symbol token(int id, String type, Object value) {
54     String lexeme = yytext();
55
56     tokens.add(new TokenInfo(
57         tokens.size() + 1,
58         lexeme,
59         type,
60         yyline + 1,
61         yycolumn + 1
62     ));
63
64     return new Symbol(
65         id,
66         yyline,
67         yycolumn,
68         value
69     );
70 }
71
72 private Symbol integerToken() {
73
74     try {
75         Integer value = Integer.valueOf(yytext());
76         return token(sym.INT, "Integer", value);
77     } catch (NumberFormatException ex) {
78
79         errors.add(new CompileError(
80             "Léxico",
81             "Número entero fuera de rango: " + yytext(),
82             yyline + 1,
83             yycolumn + 1
84         ));
85
86         return token(sym.INT, "Integer", 0);
87     }
88 }
89
90 private Symbol floatToken() {
91
92     try {
93         Double value = Double.valueOf(yytext());
94         return token(sym.FLOAT, "Float", value);
95     } catch (NumberFormatException ex) {
96
97         errors.add(new CompileError(

```

Output - OLC1\_Proyecto1\_20220306 (run) x

```

"mage"           { return token(sym.MAGE, "MAGE"); }
"warrior"        { return token(sym.WARRIOR, "WARRIOR"); }
"match"          { return token(sym.MATCH, "MATCH"); }
"main"           { return token(sym.MAIN, "MAIN"); }
"run"            { return token(sym.RUN, "RUN"); }
"with"           { return token(sym.WITH, "WITH"); }

"initial"        { return token(sym.INITIAL, "INITIAL"); }
"rules"          { return token(sym.RULES, "RULES"); }
"if"             { return token(sym.IF, "IF"); }
"then"           { return token(sym.THEN, "THEN"); }
"else"           { return token(sym.ELSE, "ELSE"); }

"players"        { return token(sym.PLAYERS, "PLAYERS"); }
"rounds"         { return token(sym.ROUNDS, "ROUNDS"); }
"scoring"        { return token(sym.SCORING, "SCORING"); }
"bonuses"        { return token(sym.BONUSES, "BONUSES"); }
"seed"           { return token(sym.SEED, "SEED"); }

"damage_point"   { return token(sym.DAMAGE_POINT, "DAMAGE_POINT"); }
"healing_point"  { return token(sym.HEALING_POINT, "HEALING_POINT"); }
"successful_defense" { return token(sym.SUCCESSFUL_DEFENSE, "SUCCESSFUL_DEFENSE"); }
"victory_bonus"  { return token(sym.VICTORY_BONUS, "VICTORY_BONUS"); }
"failed_action_penalty" { return token(sym.FAILED_ACTION_PENALTY, "FAILED_ACTION_PENALTY"); }

"mage_combo"     { return token(sym.MAGE_COMBO, "MAGE_COMBO"); }
"mage_combo_points" { return token(sym.MAGE_COMBO_POINTS, "MAGE_COMBO_POINTS"); }
"warrior_combo"  { return token(sym.WARRIOR_COMBO, "WARRIOR_COMBO"); }
"warrior_combo_points" { return token(sym.WARRIOR_COMBO_POINTS, "WARRIOR_COMBO_POINTS"); }
"low_health_victory" { return token(sym.LOW_HEALTH_VICTORY, "LOW_HEALTH_VICTORY"); }

"round_number"   { return token(sym.ROUND_NUMBER, "ROUND_NUMBER"); }
"total_rounds"   { return token(sym.TOTAL_ROUNDS, "TOTAL_ROUNDS"); }
"self_health"    { return token(sym.SELF_HEALTH, "SELF_HEALTH"); }
"opponent_health" { return token(sym.OPPONENT_HEALTH, "OPPONENT_HEALTH"); }
"self_resource"  { return token(sym.SELF_RESOURCE, "SELF_RESOURCE"); }
"opponent_resource" { return token(sym.OPPONENT_RESOURCE, "OPPONENT_RESOURCE"); }
"self_score"     { return token(sym.SELF_SCORE, "SELF_SCORE"); }
"opponent_score" { return token(sym.OPPONENT_SCORE, "OPPONENT_SCORE"); }
"self_history"   { return token(sym.SELF_HISTORY, "SELF_HISTORY"); }
"opponent_history" { return token(sym.OPPONENT_HISTORY, "OPPONENT_HISTORY"); }
"random"         { return token(sym.RANDOM, "RANDOM"); }

"get_move"       { return token(sym.GET_MOVE, "GET_MOVE"); }

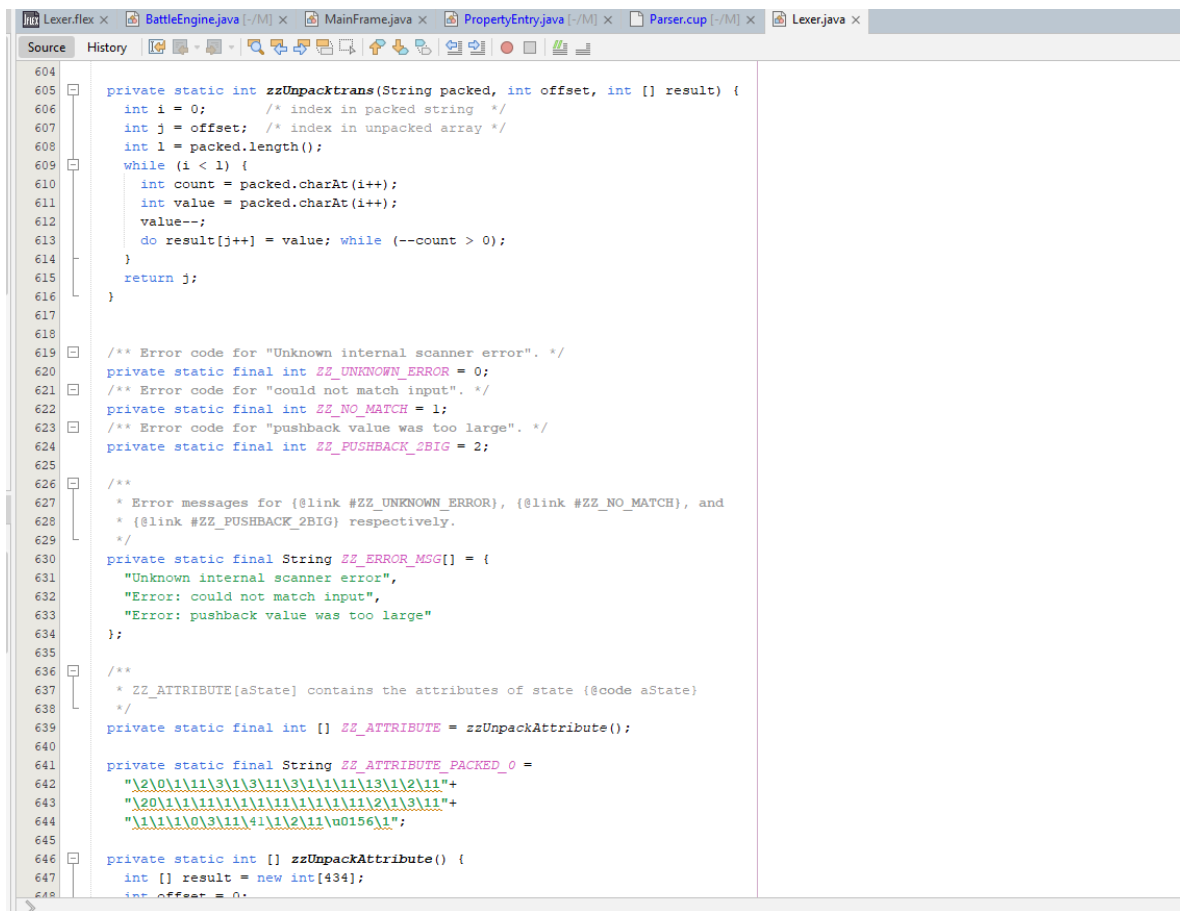
```

En las siguientes 3 capturas, podemos observar que pertenecen al Lexer.flex, el cual es nuestro analizador léxico y podemos observar que contiene nuestras palabras reservadas como:

Self\_health

Opponent\_health

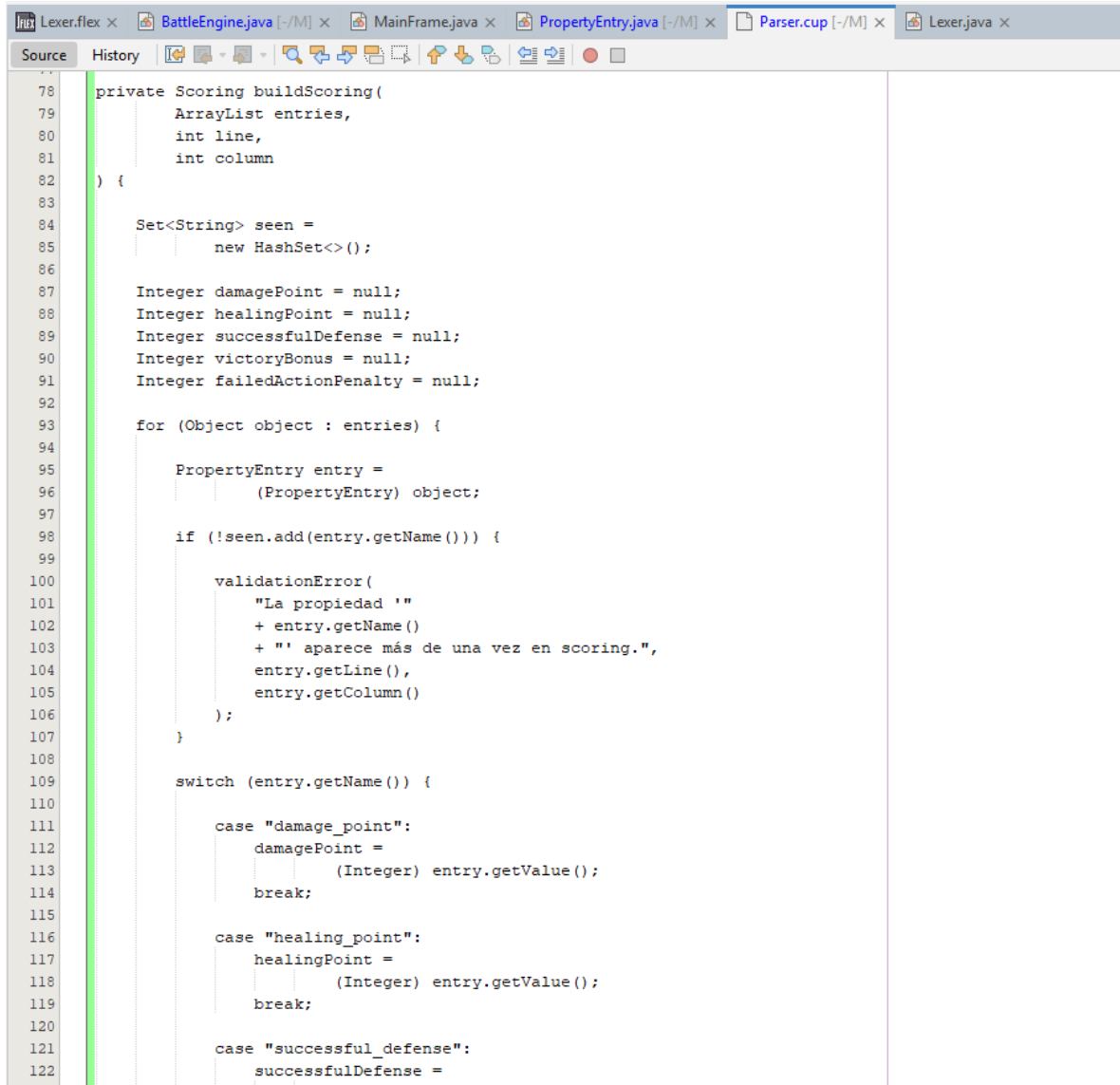
Self\_resource



```
604
605 private static int zzUnpacktrans(String packed, int offset, int [] result) {
606     int i = 0; /* index in packed string */
607     int j = offset; /* index in unpacked array */
608     int l = packed.length();
609     while (i < l) {
610         int count = packed.charAt(i++);
611         int value = packed.charAt(i++);
612         value--;
613         do result[j++] = value; while (--count > 0);
614     }
615     return j;
616 }
617
618
619 /** Error code for "Unknown internal scanner error". */
620 private static final int ZZ_UNKNOWN_ERROR = 0;
621 /** Error code for "could not match input". */
622 private static final int ZZ_NO_MATCH = 1;
623 /** Error code for "pushback value was too large". */
624 private static final int ZZ_PUSHBACK_2BIG = 2;
625
626 /**
627  * Error messages for {@link #ZZ_UNKNOWN_ERROR}, {@link #ZZ_NO_MATCH}, and
628  * {@link #ZZ_PUSHBACK_2BIG} respectively.
629  */
630 private static final String ZZ_ERROR_MSG[] = {
631     "Unknown internal scanner error",
632     "Error: could not match input",
633     "Error: pushback value was too large"
634 };
635
636 /**
637  * ZZ_ATTRIBUTE[aState] contains the attributes of state {@code aState}
638  */
639 private static final int [] ZZ_ATTRIBUTE = zzUnpackAttribute();
640
641 private static final String ZZ_ATTRIBUTE_PACKED_0 =
642     "\2\0\1\1\1\3\1\3\1\3\1\1\1\1\3\1\2\1\1"+
643     "\20\1\1\1\1\1\1\1\1\1\1\1\1\1\2\1\1\3\1\1"+
644     "\1\1\1\1\0\3\1\1\4\1\2\1\1\0\156\1";
645
646 private static int [] zzUnpackAttribute() {
647     int [] result = new int[434];
648     int offset = 0;
```

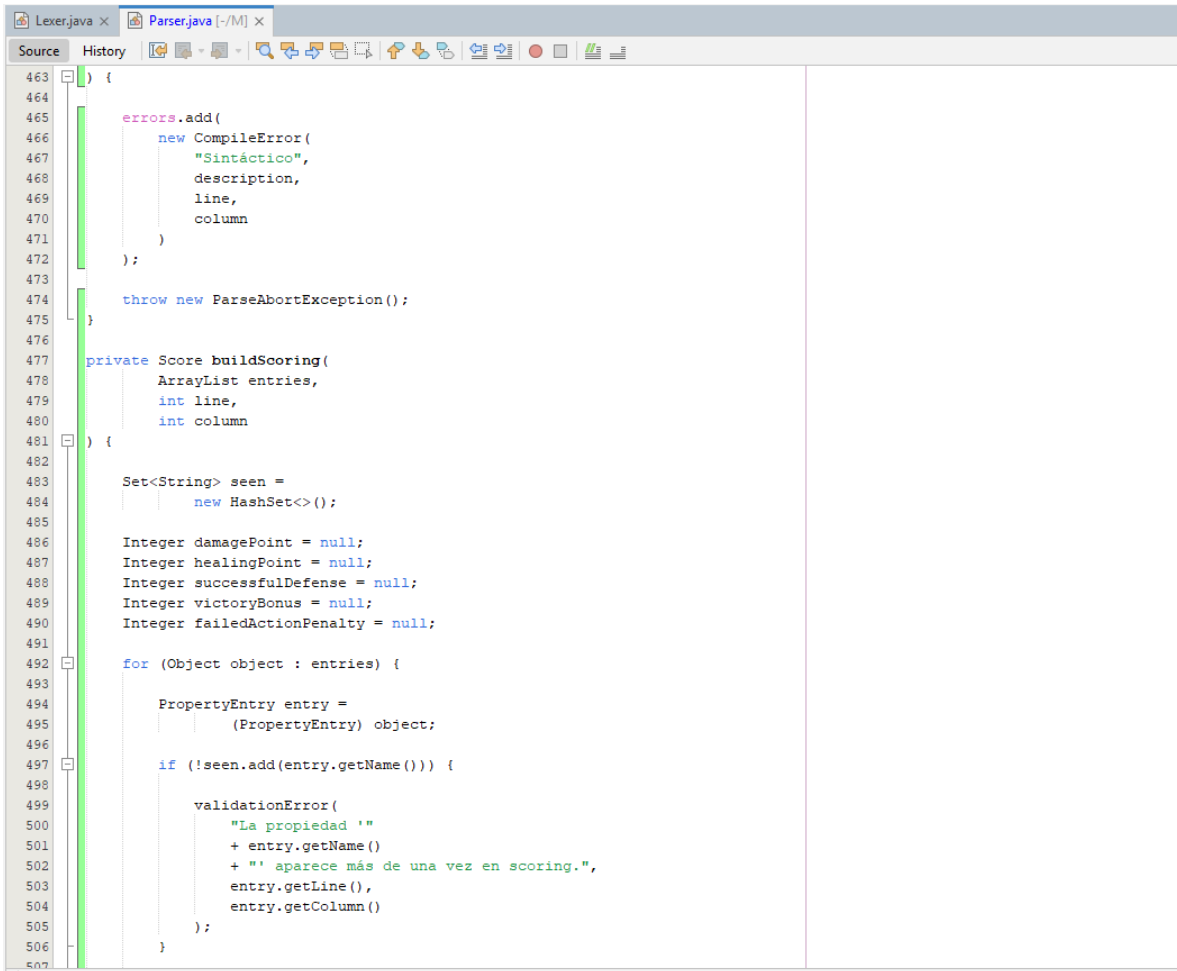
La siguiente captura pertenece al Lexer.java que este mismo se origina al momento de ejecutar nuestro archivo.bat

Este siguiente es nuestro Parser.cup, el cual es nuestro analizador sintáctico que básicamente este se encarga de verificar en el orden que vienen los tokens, si dicho orden no se respeta el programa marcará un error.



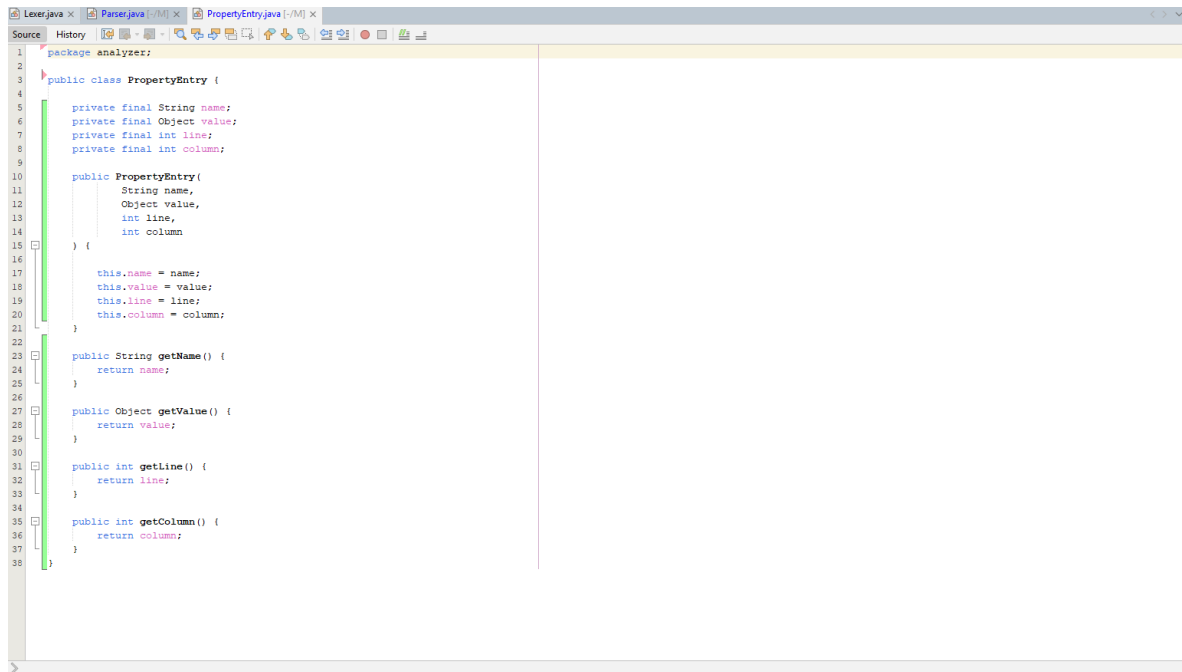
```
78 private Scoring buildScoring(  
79     ArrayList entries,  
80     int line,  
81     int column  
82 ) {  
83  
84     Set<String> seen =  
85         new HashSet<>();  
86  
87     Integer damagePoint = null;  
88     Integer healingPoint = null;  
89     Integer successfulDefense = null;  
90     Integer victoryBonus = null;  
91     Integer failedActionPenalty = null;  
92  
93     for (Object object : entries) {  
94  
95         PropertyEntry entry =  
96             (PropertyEntry) object;  
97  
98         if (!seen.add(entry.getName())) {  
99  
100             validationError(  
101                 "La propiedad '"  
102                 + entry.getName()  
103                 + "' aparece más de una vez en scoring.",  
104                 entry.getLine(),  
105                 entry.getColumn()  
106             );  
107         }  
108  
109         switch (entry.getName()) {  
110  
111             case "damage_point":  
112                 damagePoint =  
113                     (Integer) entry.getValue();  
114                 break;  
115  
116             case "healing_point":  
117                 healingPoint =  
118                     (Integer) entry.getValue();  
119                 break;  
120  
121             case "successful_defense":  
122                 successfulDefense =
```

La siguiente captura pertenece al Parser.java que este mismo se origina al momento de ejecutar nuestro archivo.bat



The screenshot shows an IDE window with two tabs: 'Lexer.java' and 'Parser.java [-/M]'. The 'Parser.java' tab is active, displaying Java code. The code is partially obscured by a vertical green bar on the left side, which appears to be a UI element or a bug. The visible code includes a method that adds a 'CompileError' to an 'errors' list and throws a 'ParseException'. It also includes a private method 'buildScoring' that initializes a 'Set' and a 'HashSet', and then iterates over a list of 'PropertyEntry' objects to validate their names. The error message in the code is: 'La propiedad ' + entry.getName() + ' aparece más de una vez en scoring.'.

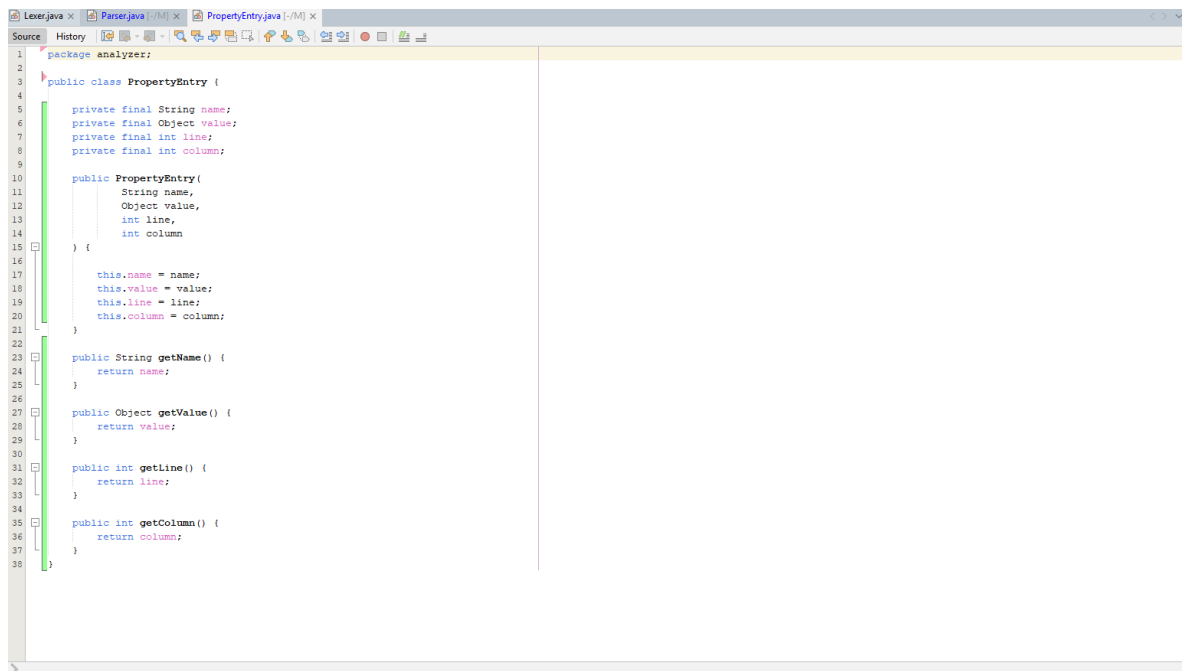
```
463 } {
464
465     errors.add(
466         new CompileError(
467             "Sintáctico",
468             description,
469             line,
470             column
471         )
472     );
473
474     throw new ParseException();
475 }
476
477 private Score buildScoring(
478     ArrayList entries,
479     int line,
480     int column
481 ) {
482
483     Set<String> seen =
484         new HashSet<>();
485
486     Integer damagePoint = null;
487     Integer healingPoint = null;
488     Integer successfulDefense = null;
489     Integer victoryBonus = null;
490     Integer failedActionPenalty = null;
491
492     for (Object object : entries) {
493
494         PropertyEntry entry =
495             (PropertyEntry) object;
496
497         if (!seen.add(entry.getName())) {
498
499             validationError(
500                 "La propiedad '"
501                 + entry.getName()
502                 + "' aparece más de una vez en scoring.",
503                 entry.getLine(),
504                 entry.getColumn()
505             );
506         }
507     }
```



```
1 package analyzer;
2
3 public class PropertyEntry {
4     private final String name;
5     private final Object value;
6     private final int line;
7     private final int column;
8
9     public PropertyEntry(
10         String name,
11         Object value,
12         int line,
13         int column
14     ) {
15         this.name = name;
16         this.value = value;
17         this.line = line;
18         this.column = column;
19     }
20
21     public String getName() {
22         return name;
23     }
24
25     public Object getValue() {
26         return value;
27     }
28
29     public int getLine() {
30         return line;
31     }
32
33     public int getColumn() {
34         return column;
35     }
36 }
37
38 }
```

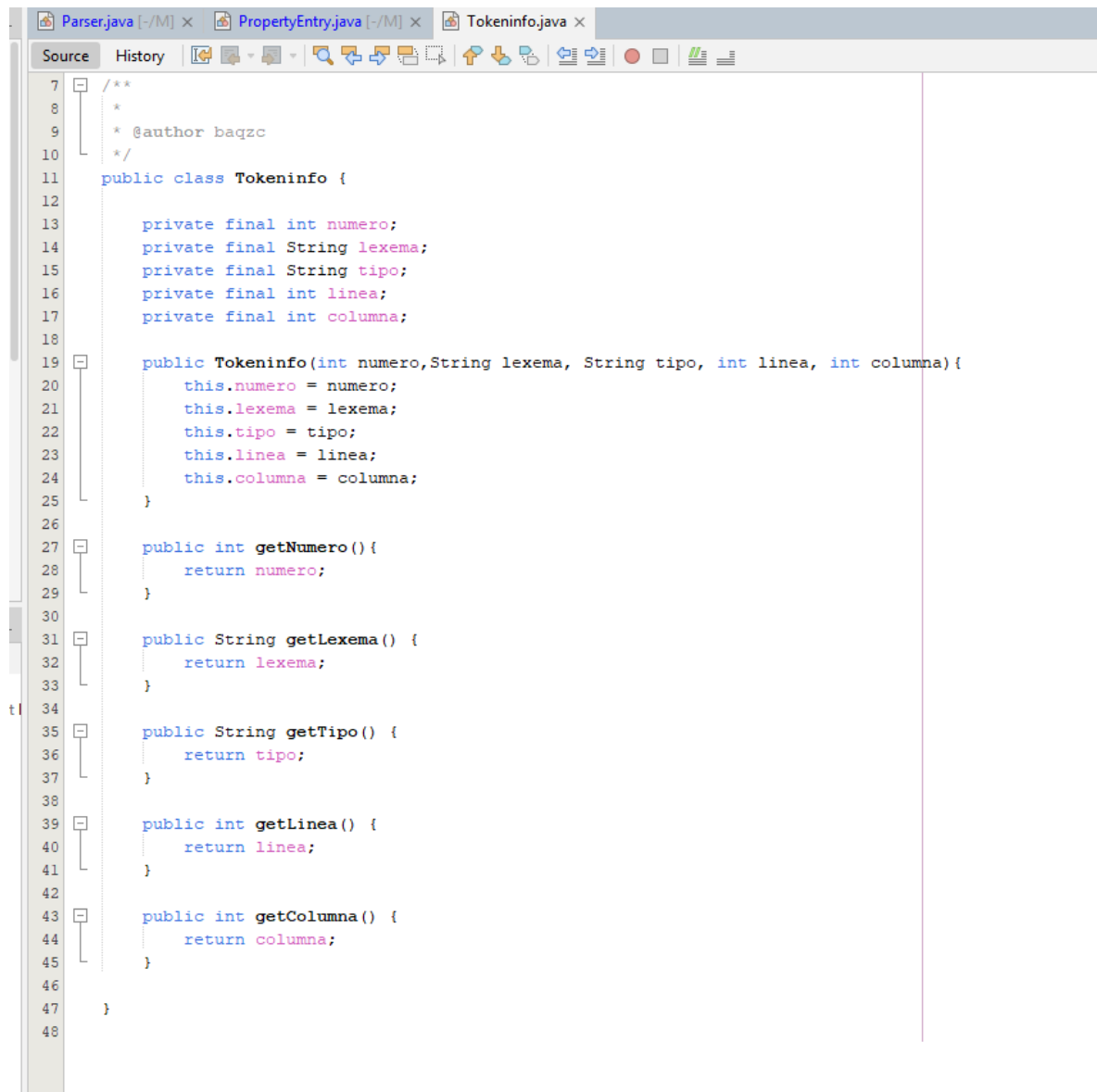
La siguiente captura tiene nuestra clase PropertyEntry.java

Que al momento de leer el archivo btl, lo que hará es leerlo línea por línea y le asignará su nombre, valor y a la columna que pertenece este mismo.



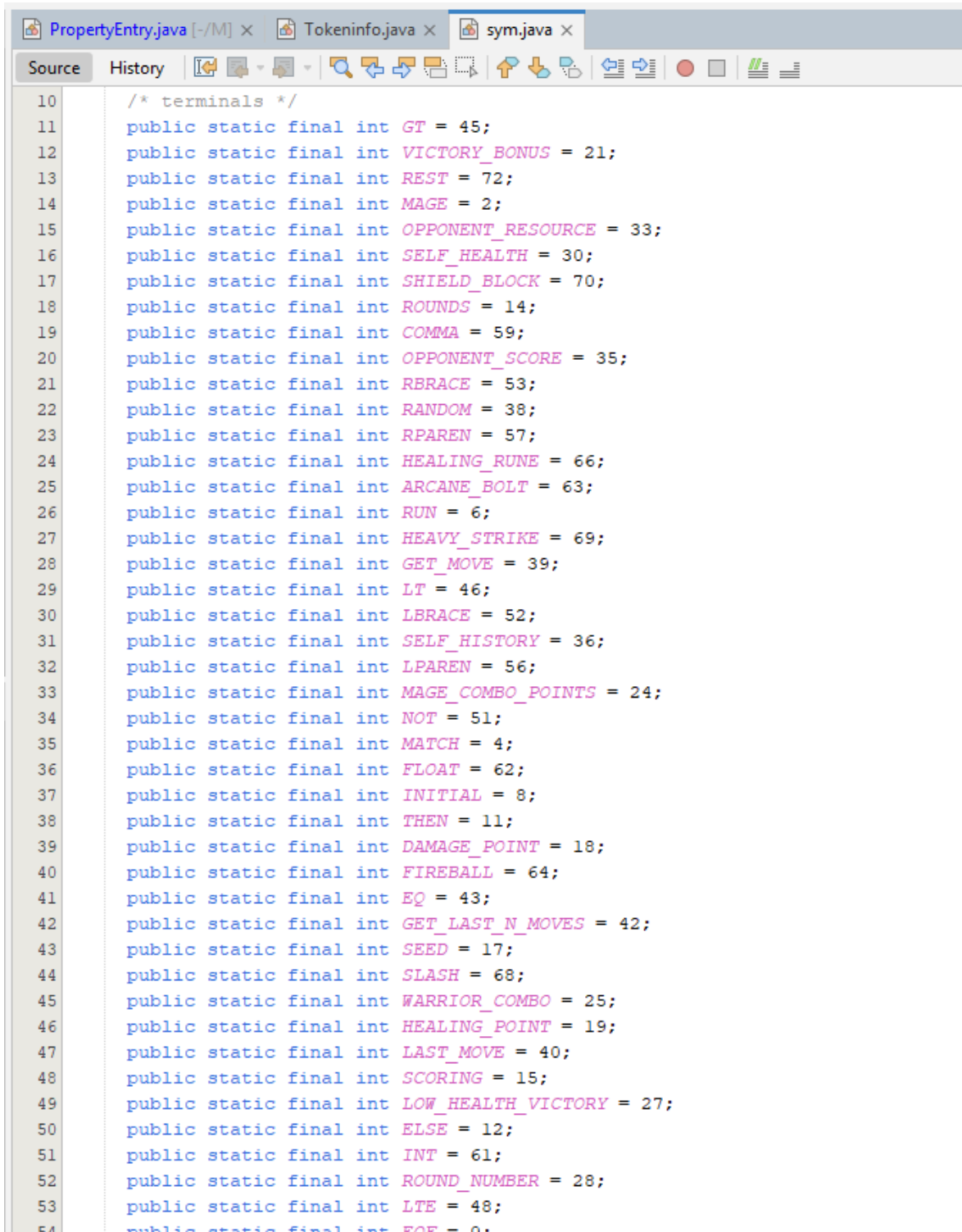
```
1 package analyzer;
2
3 public class PropertyEntry {
4     private final String name;
5     private final Object value;
6     private final int line;
7     private final int column;
8
9     public PropertyEntry(
10         String name,
11         Object value,
12         int line,
13         int column
14     ) {
15         this.name = name;
16         this.value = value;
17         this.line = line;
18         this.column = column;
19     }
20
21     public String getName() {
22         return name;
23     }
24
25     public Object getValue() {
26         return value;
27     }
28
29     public int getLine() {
30         return line;
31     }
32
33     public int getColumn() {
34         return column;
35     }
36 }
37
38 }
```

La siguiente captura nos presenta el Tokeninfo.java, la cual pues contiene la información de nuestros Tokens, básicamente lo que hará compararlos con nuestros Tokens ya registrados anteriormente y obtendrá su número, lexema, tipo, línea y columna.



```
7  /**
8   *
9   * @author baqzc
10  */
11  public class Tokeninfo {
12
13      private final int numero;
14      private final String lexema;
15      private final String tipo;
16      private final int linea;
17      private final int columna;
18
19      public Tokeninfo(int numero,String lexema, String tipo, int linea, int columna){
20          this.numero = numero;
21          this.lexema = lexema;
22          this.tipo = tipo;
23          this.linea = linea;
24          this.columna = columna;
25      }
26
27      public int getNumero(){
28          return numero;
29      }
30
31      public String getLexema() {
32          return lexema;
33      }
34
35      public String getTipo() {
36          return tipo;
37      }
38
39      public int getLinea() {
40          return linea;
41      }
42
43      public int getColumna() {
44          return columna;
45      }
46
47  }
48
```

Acá en el Sym.java es el valor de las habilidades, cuanto vida tienen o cuanto daño pueden realizar.



```
10  /* terminals */
11  public static final int GT = 45;
12  public static final int VICTORY_BONUS = 21;
13  public static final int REST = 72;
14  public static final int MAGE = 2;
15  public static final int OPPONENT_RESOURCE = 33;
16  public static final int SELF_HEALTH = 30;
17  public static final int SHIELD_BLOCK = 70;
18  public static final int ROUNDS = 14;
19  public static final int COMMA = 59;
20  public static final int OPPONENT_SCORE = 35;
21  public static final int RBRACE = 53;
22  public static final int RANDOM = 38;
23  public static final int RPAREN = 57;
24  public static final int HEALING_RUNE = 66;
25  public static final int ARCANE_BOLT = 63;
26  public static final int RUN = 6;
27  public static final int HEAVY_STRIKE = 69;
28  public static final int GET_MOVE = 39;
29  public static final int LT = 46;
30  public static final int LBRACE = 52;
31  public static final int SELF_HISTORY = 36;
32  public static final int LPAREN = 56;
33  public static final int MAGE_COMBO_POINTS = 24;
34  public static final int NOT = 51;
35  public static final int MATCH = 4;
36  public static final int FLOAT = 62;
37  public static final int INITIAL = 8;
38  public static final int THEN = 11;
39  public static final int DAMAGE_POINT = 18;
40  public static final int FIREBALL = 64;
41  public static final int EQ = 43;
42  public static final int GET_LAST_N_MOVES = 42;
43  public static final int SEED = 17;
44  public static final int SLASH = 68;
45  public static final int WARRIOR_COMBO = 25;
46  public static final int HEALING_POINT = 19;
47  public static final int LAST_MOVE = 40;
48  public static final int SCORING = 15;
49  public static final int LOW_HEALTH_VICTORY = 27;
50  public static final int ELSE = 12;
51  public static final int INT = 61;
52  public static final int ROUND_NUMBER = 28;
53  public static final int LTE = 48;
54  public static final int EOF = 0;
```

